Action Model Learning with Guarantees

Claudio Metelli and Mattia Cherubini

A.Y. 2024/2025

Abstract

In this work we will present the problem approach, the design, the implementation and evaluation of the Action Model Learning algorithm (Aineto and Scala 2024), which aims to create *sound* and *complete* PDDL models starting from a set of demonstrations.

1 Introduction

Action model engineering, which is fundamental to model-based reasoning and automatic planning in Artificial Intelligence, has historically been one of the main bottlenecks due to its complexity and intrinsic propensity for error. *Action Model Learning* (AML) addresses this problem by automating the acquisition of domain dynamics, computing an approximation of them from demonstrations (executions) of the agent.

1.1 General Idea and Utility

This work is based on an online learning algorithm that embodies the theory of AML based on version spaces. This approach interprets the learning task as a search for all hypotheses consistent with the training data, i.e., the observed transitions from state s to state s' through action a: $\langle s, a, s' \rangle$. The algorithm dynamically maintains a compact representation of the entire set of action models that could describe the agent's behaviour.

The intrinsic utility of this methodology lies in its ability to construct a rigorous and formally grounded model. Direct analysis of transitions allows the preconditions and effects of actions to be inferred, enabling the observer to deduce directly and formally what the agent is capable of doing, and likewise what it cannot do, simply by observing its movement. Furthermore, thanks to its theoretical derivation, the framework is intrinsically oriented towards providing formal guarantees and bounds on the results of the learned model. This feature makes it an ideal element to be paired with more powerful but less accurate models, such as *Large Language Models* (LLMs), as already shown by Katz et al. 2024. In a hybrid context, the rigorous model could act as a formal verification mechanism for the actions suggested by the LLM, increasing the overall reliability and transparency of the system.

1.2 Sound and Complete Models

The set of solutions maintained by the algorithm includes, at its theoretical extremes, two fundamental classes of models that approximate the true transition system of the domain: the **Sound Model** and the **Complete Model**.

Sound Model A model $\mathcal{M}$ is defined *sound* with respect to the true model $\mathcal{A}$ if every valid transition in $\mathcal{M}$ is guaranteed to be valid in $\mathcal{A}$ as well. Intuitively, it is a "safe" model which, although it may discard plans that would be valid in the true model, ensures that every plan generated and deemed valid by the sound model will certainly work in the real domain. This makes it ideal for scenarios where safety and correctness of execution are of the highest priority.

Complete Model A model $\mathcal{M}$ is defined *complete* if every valid transition in the true model $\mathcal{A}$ is also included in $\mathcal{M}$. In intuitive terms, a complete model is able to recognize the existence of a plan if it exists in the true domain: in contrast to the soundness property, the completeness property represents an optimistic view. Due to its inclusive nature, a plan generated by a complete model is not generally guaranteed to be actually executable in the real model, as it may include transitions that the agent cannot actually perform.

The combination and analysis of these two formulations allow the agent to have a rich understanding of its domain from the early stages of the learning process.

2 Action Model Learning

The Action Model Learning (AML) project is based on Mitchell's Learning by Search paradigm, developing a theory based on Version Spaces ($\mathcal{V}$). This approach interprets AML as a search for all hypotheses consistent with the training set of observed transitions. The Version Space $\mathcal{V}$ provides a compact representation of the complete set of action models consistent with the evidence. To address the computational intractability of explicitly manipulating $\mathcal{V}$, the formalism focuses on its extreme boundaries: the Lower Bound ($\mathcal{L}$), which realises the Sound Model, and the Upper Bound ($\mathcal{U}$), which realises the Complete Model. These bounds establish a partial ordering ($\subseteq$) in the hypothesis space, constraining the true action model $\mathcal{A}$:

$$\mathcal{L} \subseteq \mathcal{A} \subseteq \mathcal{U}$$

The algorithm maintains and updates these two bounds, using the set of proofs, called D, which is composed of various observations $d = \langle s, a, s' \rangle$.

Initialization and Update of Boundaries

The Action Model $A(a)$ for each action a is described by the sets of **Preconditions** Pre(a) and **Effects** Eff(a). The update is incremental at each new transition $\langle s, a, s' \rangle$.

1. Initialization

In the absence of evidence for action a, we use the set of all literals of the problem (L) and the empty set ($\emptyset$), in order to define the initial bounds, which are defined as follows.

Preconditions

$$\mathcal{L}(\text{Pre}(a)) = \{L\}, \qquad \mathcal{U}(\text{Pre}(a)) = \{\emptyset\}$$

Intuitively, with regard to preconditions, let us assume that the worst case scenario (the lower bound $\mathcal{L}$) is that action a requires all possible literals to be executed; conversely, let us assume that in the best case scenario ($\mathcal{U}$), it does not require any preconditions.

Effects

$$\mathcal{L}(\text{Eff}(a)) = \{\emptyset\}, \qquad \mathcal{U}(\text{Eff}(a)) = \{L\}$$

The effects, on the other hand, behave in the opposite way: we assume that in the worst case ($\mathcal{L}$) the action a has no effect, while in the best case ($\mathcal{U}$) the action causes the effect of changing every available literal.

2. Updating with a new Transition

Let $L(s)$ be the set of literals in state s, $d = \langle s, a, s' \rangle$ a positive demonstration and $d = \langle s, a, \bot \rangle$ a negative demonstration. In order to update a bound the following rules are applied.

Preconditions Lower Bound We update the lower bound of preconditions for action a each time an action a is executed with positive result. This indicates that every time we encounter a positive example, we can narrow down the range of all possible preconditions that must be satisfied for that action, intersecting the current lower bound with all literals given by state s:

$$\mathcal{L}(\text{Pre}(a)) \leftarrow \mathcal{L}(\text{Pre}(a)) \cap L(s).$$

Preconditions Upper Bound The upper bound of preconditions for action a is updated using negative demonstrations. When considering negative demostrations, we need to consider the set of possible hypotesis that we already have: if one hypotesis is a subset of the current state, it means that the hypotesis is too much optimistic, and so we need to enlarge the hypotesis. On the other side, if the hypotesis is not a subset, we can keep it as it is. In the case we need to enlarge the hypotesis $h_{\mathcal{U}} \in \mathcal{U}(\text{Pre}(a))$, the idea is the following:

$$h_{\mathcal{U}} \leftarrow \{h_{\mathcal{U}} \cup \{l\} \mid h_{\mathcal{U}} \in \text{Pre(a)} \wedge h_{\mathcal{U}} \subseteq s \wedge l \in h_{\mathcal{L}} \setminus s\}$$

Effects Lower Bound The Lower Bound of effects, $\mathcal{L}(\text{Eff}(a))$, is defined as the set of effects h_L that are guaranteed to occur. It must consist at least of the effects observed by the actual action, which are the literals that change from s to s'. The update rule for the Lower Bound of effects is defined as the union of the previous bound with the set of effects that become true in s' ($s' \setminus s$). This ensures that only effects consistently observed remain:

$$\mathcal{L}(\text{Eff}(a)) \leftarrow \mathcal{L}(\text{Eff}(a)) \cup L(s' \setminus s)$$

Effects Upper Bound The Upper Bound of effects, $\mathcal{U}(\text{Eff}(a))$, is defined as the set of effects h_U that are possible for action a. This set must contain all observed effects. The update rule is given by the intersection of the previous bound with the literals that are actually true in s', ensuring that the set of possible effects grows to include any new observed addition:

$$\mathcal{U}(\text{Eff}(a)) \leftarrow \mathcal{U}(\text{Eff}(a)) \cap L(s')$$

2.1 From Theoretical Approach to Practical Approach

One of the main challenges in applying Version Space theory lies in its implementation in a formal planning language, specifically the *Planning Domain Definition Language* (PDDL), which introduces the problem of **Grounding Complexity**. While in Aineto and Scala 2024 we can observe the formalism at the theoretical level, handling abstract fluents and literals, in PDDL formalization these concepts are translated into parametric predicates.

For example, what we consider a literal single l in the cited article, at implementation level it is a predicate with specific parameters. For example, if we got a predicate 'on', with parameters x and y, if there are objects o_1, o_2 and o_3 in the scene, we produce 18 possible literals: $\text{on}(o_1, o_1)$, $\text{on}(o_1, o_2)$, $\text{on}(o_1, o_3)$, etc., and their negative version.

In general, for a predicate p with N^p_{args} arguments in a domain with N^p_{objs} distinct objects, the total number of grounded literals defining the predicate space of p , called N^p_L, is given by:

$$N^p_L = 2 \cdot (N^p_{\text{objs}})^{N^p_{\text{args}}}$$

This implies that AML does not operate on the predicate in the broad sense, but on every possible instantiated literal of it. During the implementation phase, the possible precondition or possible effect must be determined with a process which is the opposite of grounding: starting from the grounded predicates, we need to obtain the symbolic ones, in order to apply the previously seen operations.

More details and examples will be provided in section 3.

3 Implementation

This section describes the implementation of the Action Model Learning approach and the following construction of models that satisfies the requirements of **soundness** and **completeness**.

The primary goal of the work is to translate what has been said in Section 2 about theory of hypothesis sets, into a software system capable of accurately gather PDDL domain models from demonstrations.

The implementation presented will focus on three fundamental logical phases associated to different functions: the procedure for creating an appropriate dataset through the Algorithm 1, the central AML module to implement the algorithm and the phase of evaluation and export of the models.

3.1 Dataset creation

The process described aims to generate a state-action transition model starting from a PDDL problem. To this end, the Unified Planning library (Micheli et al. 2025) is used to manage the process, implementing a heuristic aimed at efficiently exploring states and mantaining good sound preconditions.
To generate the dataset, we want to avoid a random selection of actions, which would result in inefficient exploration of the state space and a collection of low-quality data. Alternatively, we adopt an exploration heuristic based on fluent coverage.

The fundamental idea of the heuristic is to explore states that are considered useful in reducing uncertainty about the preconditions of the actions in the domain.

We define, for each action, the *Useful Preconditions*, which is the set of fluents grounded with every possible action parameter that are not explicitly present in the action precondition. To guide the exploration, a **heuristic score** is used, quantifying the effectiveness of each possible action in progressing toward minimizing the set of useful preconditions to be covered. Specifically, the score of an applicable action a, along with its parameters, in a state s, is defined as the number of useful preconditions that are true in the state s', following the transition $\langle s, a, s' \rangle$, and that had not yet been covered previously.

The pseudocode of Algorithm 1 will be presented in Appendix A.

Main cycle of the heuristic The heuristic is carried out through an iterative cycle that continues until one of two termination conditions is met: coverage of all useful preconditions of each action or achievement of the target state.

During each iteration, the goal is to evaluate which action is the most informative for exploration. To calculate the score assigned to each possible action, each action applicable to the current state is simulated to determine the resulting next state. The score of an action will be determined by the actual number of symbolic groundings present in the simulated next state, provided that these groundings represent useful preconditions.

Selection, Execution and Registration Once the scores have been calculated, the action with the highest score among those applicable is selected. If no action contributes to covering new fluents that are not required, a fallback procedure is implemented: a planner is executed from the current state to determine the next step toward the goal. The selected action is then applied. The symbolic groundings of the fluents that have been covered in the state reached are removed from the set of useful preconditions to explore. At this point, the transition $\langle s, a, s' \rangle$ is recorded as a positive example in the dataset. At the same time, a set of actions that are not applicable from the current state are randomly collected and recorded as negative examples.

Process Termination If the cycle ends with complete coverage of the unclaimed fluents, the planner is run to find a complete plan from the current state to the goal. All actions that make up this plan are executed in sequence, and the related transitions are recorded in the dataset as additional positive and negative examples, thus completing the dataset generation process.

3.2 Action Model Learning

This module implements the actual algorithm, based on the transition dataset that was created using the method described in Section 3.1 above. The main objective is to infer the preconditions and effects of actions in a PDDL domain by analyzing positive and negative demonstrations in the dataset.
The algorithm uses an approach that tracks both Lower Bounds ($\mathcal{L}$) and Upper Bounds ($\mathcal{U}$) of the preconditions and effects of each action.

The initialization of the four boundary sets follows this approach: the $\mathcal{L}(\text{Pre}(a))$ and the $\mathcal{U}(\text{Eff}(a))$ are initialized with the entire literal space for each action. This represents, respectively, the most specific hypothesis for the necessary preconditions and the most general hypothesis for the possible effects. Conversely, the $\mathcal{U}(\text{Pre}(a))$ and the $\mathcal{L}(\text{Eff}(a))$ are initialized to empty sets, reflecting the most general hypothesis for preconditions and the most specific hypothesis for certain effects.

Inference from positive and negative examples The first processing cycle focuses on successfully completed dataset transitions.

- $\mathcal{L}(\text{Pre}(a))$: the set is narrowed down through an intersection operation. With each success, the literals in the initial state are identified and symbolized with action parameters; the intersection of these literals with the current lower bound ensures that only predicates that are always true in every state prior to a successfull execution remain within the bound.
- $\mathcal{L}(\text{Eff}(a))$: these are expanded with an union operation. The algorithm identifies the observed state changes, symbolizes them with action parameters and adds them by union to the lower bound of the effects, keeping the accumulation of all literals that have manifested as effects at least once during successfull execution.
- $\mathcal{U}(\text{Eff}(a))$: these are narrowed by intersection on the symbolic literals of the next state, defining the maximum boundaries of possible effects and eliminating literals that are not consistently present after the action.

Subsequently, the algorithm focuses on cases of failure. This process refines the set $\mathcal{U}(\text{Pre}(a))$; this bound is handled as a set of hypotheses.

If the result of an action is negative, then it is verified that the available hypotheses are subsets of the ground state with the action parameters. The hypotheses that verify this condition certainly need to be expanded, and this expansion occurs via the lower bound, which offers a scenario within which the set of real preconditions is hidden with absolute certainty. Subsequently, the bound is reduced in order to avoid hypotheses that are subsets of others, as the upper bound needs to remain as generic as possible

Sound Model The Sound Model is designed to be restrictive and reliable. It is constructed exclusively using the two Lower Bounds ($\mathcal{L}(\text{Pre})(a)$ and $\mathcal{L}(\text{Eff})(a)$), which by definition represent certain and necessary facts.

- **Preconditions** are extracted from the literals present in the lower bound set of each action. Since this set is the result of the intersection of all initial states in positive transitions, we are certain that it contains all the real preconditions for an action. Each symbolic literal is formalized as a PDDL predicate and added as a precondition.
- **Effects** are extrected from the lower bound set, which was constructed as the union of all

observed effects. This set represents the facts that the action has certainly caused at least once. These literals are added as effects of the action in the new domain.

In summary, the Sound Model represents the safest and less optimistic hypothesis, including all necessary facts and certain observed effects.

Complete Model The Complete Model is designed to be more permissive and general, even at the cost of including potentially incorrect literals. It is constructed exclusively using the two Lower Bounds ($\mathcal{U}(\text{Pre})(a)$ and $\mathcal{U}(\text{Eff})(a)$).

- **Preconditions**: the upper bound of preconditions is a set of hypothesis, each of which is considered a valid alternative in the model, so that even if only one of them is satisfied, the action is considered applicable. This is translated into the PDDL model preconditions of an action through the use of the OR statement, which combines all possible hypotheses in $\mathcal{U}(\text{Pre})(a)$, verifying that even just one of them is verified. This implies that the action is applicable if at least one of these hypothesis is satisfied.

- **Effects**: in this case, we implement a logic of *Action Decomposition* from the original action. The hypothesis space of effects is calculated, covering all possible literals between the two bounds.
  The original action in the domain is replaced by a set of derived actions and each one of these a_i is assigned a discrete and deterministic set of effects that correspond to a single hypothesis in the hypothesis space of effects. This method makes the uncertainty of the effects explicit through a disjunction at the action level, ensuring maximum completeness of the model.

As highlighted by the intrinsic characteristics of soundness and completeness, the Sound Model is often considered to be of greater relevance in terms of predictive reliability with regard to inferred effects, thanks to its basis of certain facts.

4 Experimantal Evaluation

In this section, we will evaluate the performance of the AML algorithm in order to provide an objective measure of the effectiveness of the Sound and Complete models produced on a selection of planning domains, using appropriate metrics for each property.

Domains studied. We select 4 domains named `BLOCKSWORLD`, `GARDENWORLD`, `ZENO` and `GROCERY`, that present an increasing level of complexity and challenge the algorithm on different aspects, as suggested by their structure.

From these domains, specific PDDL problems were generated. These problems, together with the relevant domain and a defined number of objects, were then submitted to the main process pipeline, as described in Section 3. This phase allowed the collection of a dataset necessary for training the AML model, the final results of which are analyzed in the following sections.

Evaluation criteria. The evaluation of the learned models is performed by comparing the Sound Model derived from Lower Bounds $\mathcal{L}$ and the Complete Model derived from the Upper Bounds $\mathcal{U}$ with the true PDDL domain. Specifically, metrics are used that measure structural accuracy, i.e. how much the learned symbolic predicates overlap with those of the true model.

In order to evaluate the behaviour of the **Sound Model** we look at the intersection between the learned predicates and the real ones. For preconditions we calculate the ratio between the set of learned predicates and the real ones. Ideally, a high value confirms that the model is not introducing false positives in an attempt to generalize. Then recall measures how many real predicates have actually been captured and a low value indicates that the Sound Model is incomplete, in favor of execution safety.

For effects, on the other hand, the metric evaluates whether the lower bound of effects has captured all effects that always occour. A 100% in this measure indicates that the model has perfectly learned the deterministic effects of the domain.

The **Complete Model** is evaluated based on its ability to maximize coverage, and therefore completeness, accepting a compromise on precision.

For preconditions we have that structural precision and recall metrics are used directly, since the set is a disjunction of hypotheses.

- **Precision** measures the quality of the individual hypotheses generated i.e. the percentage of correctly predicted preconditions

among all the predicted preconditions. Although the model is optimistic, a low precision suggests that the hypotheses in the set are composed of an excessive number of predicates that do not belong to the real model.

- **Recall** measures the ability of the model to infer the predicates required by the real model i.e. the percentage of correctly predicted preconditions among the real preconditions. A high value (close to 1.0) indicats that the set of hypotheses adequately covers all the predicates necessary for the validity of the model.

For Effects, the analysis focuses on precision, measuring the ratio between common effects and total learned effects. Since the logic of the Complete Model is optimistic, it is expected that the upper bound of effects will be an overstimate of the actual set, accepting false positives and confirming its nature as a potentially unsound model.

Implicit correlations and Related predicates. A fundamental aspect that emerged from the analysis of the Lower Bound $\mathcal{L}$ is the tendency of the algorithm to learn implicit correlations or redundant predicates that are consistently true in the state, but which are not strictly necessary for the logic validity of the PDDL precondition. This event is not an error of the algorithm, but reflects the high domain dependency: the algorithm, operating by set operations on the facts of the state, captures everything that is consistently true, even if it is not essential for the logical purposes of the minimal PDDL action. For example, when the algorithm computes the Lower Bound of preconditions through the intersection operations, it includes any fact that is invariably true. These addictional facts are not required by the formal definition of the action, but are a consequence of the domain definition and their inclusion makes the *Sound Model* redundant with respect to the minimal PDDL model, but not logically incorrect.

There are specific cases to prove that, like mutual exclusion, a phenomenon which occurs when two predicates are mutually exclusive by definition of the problem, and one of them is the explicit precondition. The algorithm, by intersecting all states, captures both the action and the implicit fact and the result is that the resulting Sound Model therefore includes this last as a redundant predicate. An example is in the domain `BLOCKSWORLD` where the `put-down` action explicitly requires `(holding ?x)`. In the PDDL state, the truth of `(holding ?x)` implies that `(arm-empty)` is false. So in the Sound Model there will be `(not (arm-empty))` as redundant.

Another case which is important to include in this discussion concerns **Domain-invariant dependencies**, which occours when the specific problem imposes an invariance that is not required by the individual action. For example in the domain `GROCERY` there are predicates in this form: `(connected ?a ?b)` modeling the connections between compartments and if these are always generated symmetrically, the Lower Bound may include the symmetric form, even though the move action only requires the connection in one direction. This is a learned invariance that is not a Soundness error, but another confirm of the caution of the model.

Results. The analysis of the domains studied aims to understand under which structural conditions the algorithm achieves maximum convergence between the limits $\mathcal{L}$ and $\mathcal{U}$.
A fundamental result is the absolute precision in capturing the effects of the Sound Model. In each scenario, the Lower Bound of the effects has achieved 100% structural precision and recall, which demonstrates how the effects in the classical domains are highly deterministic and the algorithm is able to isolate the consequences of each action. Limitations can be encountered in the generalization of complete preconditions in more complex domains, and this translates into low accuracy of the complete model (confirming the theoretical tendency to overestimate and include false positives) and incomplete recall, which indicates that the Upper Bound of the preconditions has not been sufficiently enlarged with negative examples.

This phenomenon is directly related to the mechanism of generating negative examples in the dataset. As implemented, non applicable actions are collected by randomly selecting them from the set of all possible grounded actions for a given state.
The random behaviour of this process means that the negative examples may not be sufficient informative to eliminate all inconsistent hypothesis from the Upper Bound set.
So, the resulting set of Complete preconditions, fails to acheive the high coverage required, espacially in domains with many predicates with a large hypothesis space.

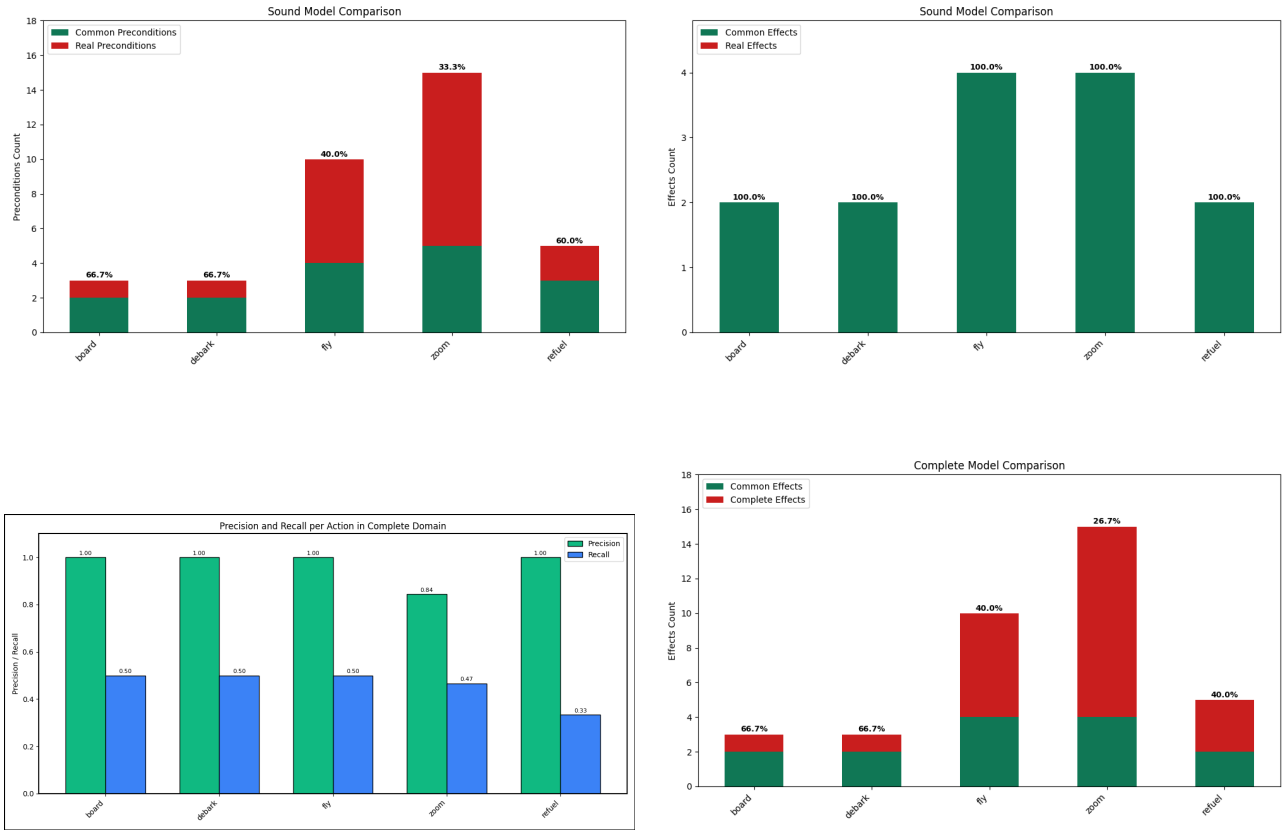

Figure 1: Actions (x-axis) and metrics (y-axis) of ZENO domain

Figure 1 shows the results obtained from the analysis of ZENO domain, which emerges as a significant case for the validation of the algorithm.

As regards the **Sound Model**, it can be seen that all the actions involved allowed the Lower Bound of effects to reach structural Recall value of 100% in all measurements. This result confirm that all necessary state changes were captured by the intersection operation. However, it can be noticed that its soundness property is confirmed by the low percentage shown in the preconditions (33.0% for the most complex actions), a result consistent with the model's safety objective.

The **Complete Model**, on the other hand, highlighted the limitations of generalization due to complex structure of its actions. Precision of effects has dropped to 26.7% and this decline confirms that the Upper Bound included a large number of false positives to mantain completeness, resulting in an unsound model. This overstimation occours because the Version Space has not been sufficiently restricted by the available actions, causing the boundary made by the Upper Bound of effects to remain too loose and include unintended consequences. Furthermore, analysis of the complete precondition metrics highlighted a crucial difficulty: despite the high Precision of individual hypotheses, overall Recall remained insufficient. This scenario suggests that the Upper Bound of preconditions is not sufficiently broad, indicating that the limitation lies in the quality of the negative sampling, which failed to restrict the version space to the area of complete coverage.

In conclusion, Zeno validates the Lower Bound's ability to learn complex deterministic logic, but at the same time highlights the inherent difficulties of the Complete Model in achieving the goal of perfect recall in a structurally rich environment, making it the most instructive domain for analyzing the trade-offs of the Version Space.

5 Conclusions

In conclusion, we presented a practical way to develop the AML algorithm, showing that significant result as been reached using this algorithm. At the same time, the best way to improve performances does not rely on the algorithm itself, but on the demonstration presented to the algorithm.

Following this idea, a simple heuristic method has been presented in order to improve accuracy

of the overall algorithm, but more changes can be added: for example, an online algorithm that combines both AML algorithm and exploration of the space following some heuristic in order to collect useful demonstrations could be a significant improvement for the overall accuracy of the model.

References

Aineto, Diego and Enrico Scala (Nov. 2024). "Action Model Learning with Guarantees". In: *Proceedings of the TwentyFirst International Conference on Principles of Knowledge Representation and Reasoning*. KR-2024. International Joint Conferences on Artificial Intelligence Organization, 801–811. DOI: 10.24963/kr.2024/75. URL: http://dx.doi.org/10.24963/kr.2024/75.

Katz, Michael et al. (2024). *Thought of Search: Planning with Language Models Through The Lens of Efficiency*. arXiv: 2404.11833 [cs.AI]. URL: https://arxiv.org/abs/2404.11833.

Micheli, Andrea et al. (2025). "Unified Planning: Modeling, manipulating and solving AI planning problems in Python". In: *SoftwareX* 29, p. 102012. ISSN: 2352-7110. DOI: https://doi.org/10.1016/j.softx.2024.102012. URL: https://www.sciencedirect.com/science/article/pii/S2352711024003820.

A Pseudocode for Demonstration Set generation

Algorithm 1 Generate Demonstration Set

FUNCTION GenerateDemonstrationSet(Problem, Domain)
$s \leftarrow$ Problem.InitialState
$G \leftarrow$ Problem.Goal
DemonstrationSet $\leftarrow \emptyset$
$\mathcal{UP} \leftarrow \emptyset$
$\mathcal{UP}$.Initialize()
while $\mathcal{UP}$ IS NOT EMPTY **AND** s IS NOT G **do**
  Scores $\leftarrow \emptyset$
  for each applicable action a in s **do**
    $\mathcal{H}(a) \leftarrow$ CalculateHeuristicScore$(a, s, \mathcal{UP})$
    Scores$[a] \leftarrow \mathcal{H}(a)$
  end for
  $\mathcal{H}_{max} \leftarrow$ max(Scores)
  $a_{\text{next}} \leftarrow$ NULL
  if $\mathcal{H}_{max} > 0$ **then**
    $a_{\text{next}} \leftarrow$ GetActionWithMaxScore(Scores)
  else
    $a_{\text{next}} \leftarrow$ Planner.FindNextAction(s, G)
  end if
  $s' \leftarrow$ Simulator.Apply(s, a_{next})
  $\mathcal{UP}$.RemoveCovered(a_{next}, s') {Update $\mathcal{UP}$ based on the observed transition}
  DemonstrationSet.ADD(PositiveExample(s, a_{next}, s'))
  DemonstrationSet.ADD_ALL(NegativeRandomExamples(s))
  $s \leftarrow s'$
end while
if s IS NOT G **then**
  RemainingPlan $\leftarrow$ Planner.FindPlan(s, G)
  for each action a in RemainingPlan **do**
    $s' \leftarrow$ Simulator.Apply(s, a)
    DemonstrationSet.ADD(PositiveExample(s, a, s'))
    DemonstrationSet.ADD_ALL(NegativeExamples(s))
    $s \leftarrow s'$
  end for
end if
RETURN DemonstrationSet
END FUNCTION